

Intern Daily Task Plan

2026-07-21 through 2026-10-01 · weekdays only · 53 working days

Starting point: complete beginner to programming. **Arc:** JS fundamentals → React → React Native/Expo → TypeScript & testing → an independent practice app → onboarding into the real **EliteMobile** codebase → supervised real contributions. Each day assumes ~1 focused task; if they finish early, have them explain what they built out loud before moving on — that catches gaps faster than moving on to more content.

Week 1 · Jul 21–24 · Programming Fundamentals I

Goal: get the toolchain working and build comfort with the absolute basics.

Date	Task	Details
Tue Jul 21	Dev environment	Finish the Expo Setup Guide (Node, Git, VS Code, Expo Go). Confirm <code>node -v</code> , <code>npm -v</code> , <code>git --version</code> all work. Create a GitHub account. Write and run a plain <code>.js</code> file with <code>node file.js</code> to prove the whole toolchain works end to end.
Wed Jul 22	JS basics I	Variables (<code>let</code> / <code>const</code>), data types, operators, template literals. Exercise: a tip calculator script that takes a bill amount and prints the tip and total.
Thu Jul 23	JS basics II	Conditionals (<code>if</code> / <code>else</code> , <code>switch</code>), comparison/boolean logic. Exercise: FizzBuzz, plus a script that turns a numeric score into a letter grade.
Fri Jul 24	JS basics III	Loops (<code>for</code> , <code>while</code>), arrays basics. Exercise: sum an array of numbers, find the max, reverse a string without using <code>.reverse()</code> .

Week 2 · Jul 27–31 · Programming Fundamentals II + Git

Goal: functions, real array/object manipulation, async basics, and version control habits.

Date	Task	Details
Mon Jul 27	Functions	Declarations vs. arrow functions, parameters/return values, scope. Exercise: refactor Week 1's scripts into reusable, named functions.
Tue Jul 28	Arrays deep dive	<code>map</code> / <code>filter</code> / <code>reduce</code> / <code>forEach</code> and when to use each. Exercise: given an array of worker objects with <code>name</code> / <code>hoursWorked</code> , compute total hours, filter anyone over 8 hours, and map to just names.

Wed Jul 29	Objects & destructuring	Object literals, nested objects, destructuring, spread/rest syntax. Exercise: model a simple job-ticket object and write functions that read/update it immutably (spread, never mutate directly).
Thu Jul 30	Async basics	Callbacks, Promises, <code>async / await</code> , <code>fetch</code> . Exercise: fetch a list of fake users from a public test API (e.g. <code>jsonplaceholder.typicode.com</code>) and print their names.
Fri Jul 31	Git & GitHub	<code>init / add / commit / push</code> , branches, pull requests, <code>.gitignore</code> . Exercise: push all of Weeks 1–2's exercises to a personal GitHub repo with an honest commit history (not one giant commit).

Week 3 · Aug 3–7 · React Fundamentals I

Goal: components, props, state — the core mental model of React.

Date	Task	Details
Mon Aug 3	What is React?	Components, JSX, why a UI library exists. Set up a web practice project (Vite + React is fine). Exercise: a static "profile card" component with hardcoded text.
Tue Aug 4	Props	Passing data into components, composing multiple components. Exercise: make the profile card reusable — render 3 different people via different props.
Wed Aug 5	State (useState)	Click counters, toggles, controlled state. Exercise: a counter app (+/- buttons) and a show/hide toggle.
Thu Aug 6	Lists & keys	Rendering arrays with <code>.map()</code> , why <code>key</code> matters. Exercise: a to-do list from a hardcoded array, each item with a working "remove" button.
Fri Aug 7	Forms	Controlled inputs, checkboxes, handling submit. Exercise: extend the to-do list with a text input + "Add" button that appends to state.

Week 4 · Aug 10–14 · React Fundamentals II

Goal: side effects, composition, and a first real mini-project checkpoint.

Date	Task	Details
Mon Aug 10	useEffect	Side effects, dependency arrays, cleanup functions. Exercise: a stopwatch/timer using <code>setInterval</code> inside <code>useEffect</code> .
Tue Aug 11	Conditional rendering	Ternaries, <code>&&</code> , early returns, loading/error/empty states. Exercise: fake a loading delay before showing the to-do list, plus an empty-state message.

Wed Aug 12	Composition	Breaking components apart, <code>children</code> , lifting state up. Exercise: refactor the to-do app into <code>TodoList</code> / <code>TodoItem</code> / <code>AddTodoForm</code> with state lifted to a parent.
Thu Aug 13	Custom hooks	Extracting reusable logic. Exercise: write a <code>useCounter</code> hook and use it in two different components.
Fri Aug 14	Checkpoint project	Build a small quiz app (multiple-choice questions, score tracking, restart button) combining everything from Weeks 3–4.

Week 5 · Aug 17–21 · React Native + Expo Fundamentals

Goal: move everything learned so far onto a real phone.

Date	Task	Details
Mon Aug 17	Web → mobile	<code>View</code> / <code>Text</code> / <code>Image</code> / <code>ScrollView</code> replace <code>div</code> / <code>span</code> / <code>img</code> . Exercise: <code>npm create-expo-app practice-app</code> , run it in Expo Go, rebuild the Day-1-of-React profile card with RN components.
Tue Aug 18	StyleSheet & Flexbox	<code>StyleSheet.create</code> , <code>flexDirection</code> / <code>justifyContent</code> / <code>alignItems</code> . Exercise: a header/body/footer layout using only Flexbox.
Wed Aug 19	Touch & input	<code>Pressable</code> , <code>TextInput</code> , <code>Alert</code> . Exercise: port the counter and to-do list into React Native components.
Thu Aug 20	Lists on mobile	<code>FlatList</code> vs. <code>.map()</code> , <code>keyExtractor</code> / <code>renderItem</code> . Exercise: render 50+ fake items with <code>FlatList</code> instead of <code>ScrollView+map</code> .
Fri Aug 21	Images & safe areas	<code>Image</code> / <code>expo-image</code> , <code>SafeAreaView</code> / <code>useSafeAreaInsets</code> . Exercise: add a header + avatar image that correctly avoids the phone's notch/status bar.

Week 6 · Aug 24–28 · Navigation, Styling Patterns, Device APIs

Goal: multi-screen apps and touching real device capabilities.

Date	Task	Details
Mon Aug 24	Expo Router basics	File-based routing, <code>app/index.tsx</code> , <code>Link</code> / <code>router.push</code> . Exercise: add a second screen and navigate to it with a route param.
Tue Aug 25	Tab & stack nav	The <code>(tabs)</code> folder convention, nested <code>_layout.tsx</code> . Exercise: turn the practice app into a 3-tab app (Home / List / Profile).

Wed Aug 26	Theming & reusable UI	Colors/spacing constants, avoiding magic numbers, small reusable atoms. Exercise: extract a <code>colors.ts</code> and a shared <code>Button</code> component used across all 3 tabs.
Thu Aug 27	Device APIs I	Permissions model, <code>expo-image-picker</code> (or <code>expo-location</code>). Exercise: add a "pick a photo from your library" button to the Profile tab.
Fri Aug 28	Device APIs II	Local persistence (AsyncStorage/MMKV). Exercise: make the to-do list survive closing and reopening the app.

Week 7 · Aug 31–Sep 4 · TypeScript + Testing Fundamentals

Goal: type safety and a first taste of automated testing.

Date	Task	Details
Mon Aug 31	TypeScript I	Why types, primitives, typing function params/returns, <code>interface</code> vs <code>type</code> . Exercise: add TypeScript to the practice app; type the to-do item shape.
Tue Sep 1	TypeScript II	Typing component props, optional props, union types, typing <code>useState</code> . Exercise: add proper prop types everywhere; fix every red squiggly.
Wed Sep 2	Why we test	Jest basics, <code>expect</code> / <code>toBe</code> / <code>toEqual</code> . Exercise: write 3 tests for a plain helper function (e.g. <code>calculateTotal</code>), no UI involved.
Thu Sep 3	Testing components	React Testing Library: <code>render</code> , <code>screen.getByText</code> , simulating a tap. Exercise: test that the counter's count increases on tap.
Fri Sep 4	Testing hooks	<code>renderHook</code> , testing a custom hook in isolation. Exercise: write tests for the <code>useCounter</code> hook from Week 4.

Week 8 · Sep 7–11 · Practice App Build (Part 1)

Goal: plan and build a small real app end to end, alone.

Date	Task	Details
Mon Sep 7	Planning day	Pick a small app idea (Habit Tracker, Recipe Box, Workout Log, etc.), sketch 3 screens on paper, write a short plan (data model, screens, user actions). Present the plan before writing any code.
Tue Sep 8	Scaffold	New Expo TypeScript project, Expo Router with the 3 planned screens as empty placeholders, a theming/colors file. Get navigation working end to end before any real content.
Wed Sep 9	Main list screen	Data model, <code>FlatList</code> of items, an "add new item" form flow.

Thu Sep 10	Detail screen	Tapping an item navigates to a detail view; add an edit or delete action.
Fri Sep 11	Persistence	Wire up AsyncStorage/MMKV so data survives an app restart; write at least one test for the core data logic.

Week 9 · Sep 14–18 · Practice App Build (Part 2) & Wrap-Up

Goal: polish, test, and present a finished, working app.

Date	Task	Details
Mon Sep 14	UI polish	Consistent spacing/colors, loading/empty states, small feedback touches (haptics or simple transitions, optional).
Tue Sep 15	Error handling	Empty input, very long text, zero-items states. Add form validation.
Wed Sep 16	Testing pass	Write tests for the 2–3 most important pieces of real logic (not just rendering).
Thu Sep 17	Code review day	Review the practice app's code together; apply feedback on naming, duplication, and structure.
Fri Sep 18	Demo day	Intern presents the finished practice app. This is the readiness checkpoint before moving into EliteMobile.

Week 10 · Sep 21–25 · Onboarding into EliteMobile

Goal: understand the real codebase before changing anything in it.

Date	Task	Details
Mon Sep 21	Get it running	Clone/open EliteMobile, install dependencies, run it via Expo Go or a simulator/emulator, read any README/CLAUDE.md notes. Click through every screen as a user would — no code reading yet.
Tue Sep 22	Read the architecture	Walk one full module end to end (e.g. <code>roster</code>): entity → usecase → adapter → viewModel → screen. Draw a hand diagram of how a tap flows to a screen update.
Wed Sep 23	Read the tests	Open a few <code>*.test.tsx</code> files; understand what's verified and how test doubles/mock dependencies are wired in.
Thu Sep 24	Trace a past fix	Pick one already-fixed, documented bug in the project and understand <i>why</i> it needed fixing by reading the surrounding code, not just the diff.
Fri Sep 25	Shadow task	Pick one small flagged cleanup item, write up a plan to fix it, and review the plan with a mentor before touching any code.

Week 11 · Sep 28–Oct 1 · Real Contributions + Wrap-Up

Goal: land real, reviewed changes in EliteMobile.

Date	Task	Details
Mon Sep 28	First real fix	Implement Friday's plan, run the existing tests, confirm nothing breaks, open a small diff/PR for review.
Tue Sep 29	Review + second task	Address review feedback on the first fix; pick up one more small, well-scoped task with mentor guidance.
Wed Sep 30	Finish second task	Implement, test, get it reviewed. Start drafting a short "what I learned" writeup.
Thu Oct 1	Wrap-up	Final review of both contributions; intern presents a recap of the practice app and the EliteMobile fixes; discuss next steps.